

SEMICONDUCTOR WAGE ANALYSIS

U.S. Semiconductor Sector — Earnings Insights

Overview

This document describes the Python analysis developed to extract wage insights from U.S. Bureau of Labor Statistics employment earnings tables, supplemented by the **Employment Cost Index (ECI)** as a benchmark for compensation trends. Eight CSV files — covering seasonally adjusted (SA) and not seasonally adjusted (NSA) weekly and hourly earnings for all employees and production workers — feed six insight functions. Together they address wage growth leadership, sector premiums, stagnation risk, skills gaps, temporal acceleration, and real purchasing power. Data spans NAICS-coded subsectors of the semiconductor and electronic component manufacturing industry.

Data Sources

Eight CSV files are loaded at the top of the script:

Variable	File	Adjustment	Scope
weekly_sa	Table B-3a average_weekly_earnings.csv	SA	All employees
weekly_nsa	Table B-3a average_weekly_earnings_not_seasonally_adjusted.csv	NSA	All employees
hourly_sa	Table B-3b average_hourly_earnings.csv	SA	All employees
hourly_nsa	Table B-3b average_hourly_earnings_not_seasonally_adjusted.csv	NSA	All employees
weekly_prod_sa	Table B-8a average_weekly_earnings_production_employees.csv	SA	Production workers
weekly_prod_nsa	Table B-8a ...not_seasonally_adjusted.csv	NSA	Production workers
hourly_prod_sa	Table B-8b average_hourly_earnings_production_employees.csv	SA	Production workers
hourly_prod_nsa	Table B-8b ...not_seasonally_adjusted.csv	NSA	Production workers

Insight 1 — Wage Growth Leadership (insight_1_communications_surge)

Objective

Rank all subsectors in weekly_sa by year-over-year weekly earnings growth from December 2024 to December 2025. The function identifies which industry posted the highest nominal wage growth over the period and surfaces the winner's absolute earnings figures.

Python Code

```
def insight_1_communications_surge():
    results = []
    for idx, row in weekly_sa.iterrows():
        naics = row['NAICS Code']
        industry = row['Industry']
        dec24 = row['Dec-24']
        dec25 = row['Dec-25']
        growth = ((dec25 - dec24) / dec24)
        results.append({'Industry': industry, 'NAICS': naics,
                        'Dec-24': dec24, 'Dec-25': dec25, 'Growth %': growth})
    df_results = pd.DataFrame(results).sort_values('Growth %', ascending=False)
    winner = df_results.iloc[0]
    return df_results
```

Key Design Choices

- A plain Python list is populated row-by-row then converted to a DataFrame in one step, avoiding repeated DataFrame concatenation which is slow in pandas.
- Growth is stored as a decimal fraction (not multiplied by 100) inside the loop; formatting to a percentage string happens only in the print statements, keeping the stored values numerically clean for downstream sorting.
- Sorting ascending=False puts the highest-growth sector at iloc[0], making winner retrieval a single line.
- The function returns the full ranked DataFrame so callers can slice any tier of the rankings without re-running the loop.

Troubleshooting Notes

An early version multiplied growth by 100 inside the loop, causing the sort to still work correctly but the 'Growth %' column to be inconsistently scaled relative to the print format string. Storing the raw fraction and only formatting at print time resolved the discrepancy.

Insight 2 — Semiconductor Device Premium (insight_2_premium_gap)

Objective

Quantify the earnings premium that semiconductor device manufacturing workers (NAICS 334413) command over the broader electronic component manufacturing average (NAICS 334)

in December 2025, and track whether that premium expanded or contracted versus December 2024.

Python Code

```
def insight_2_premium_gap():
    semi = weekly_sa[weekly_sa['NAICS Code'] == 334413].iloc[0]
    industry = weekly_sa[weekly_sa['NAICS Code'] == 334].iloc[0]
    premium_pct = ((semi['Dec-25'] - industry['Dec-25']) / industry['Dec-25']) *
100
    premium_weekly = semi['Dec-25'] - industry['Dec-25']
    premium_annual = premium_weekly * 52
    premium_24 = ((semi['Dec-24'] - industry['Dec-24']) / industry['Dec-24']) *
100
    premium_25 = premium_pct
    premium_expansion = premium_25 - premium_24
    return {'premium_annual': premium_annual,
            'premium_pct': premium_pct,
            'premium_expansion': premium_expansion}
```

Sample Result

Metric	Value
Semiconductor Device (Dec-25)	\$ / week (annualised)
Industry Average (Dec-25)	\$ / week (annualised)
Weekly Premium	\$
Annual Premium	\$
Premium % (Dec-25)	%
Premium Expansion vs Dec-24	+/- pp

Key Design Choices

- Filtering with a boolean mask and immediately calling .iloc[0] extracts a single Series, making subsequent column references clean and readable.
- Annualising by multiplying weekly premium by 52 translates the advantage into a figure that is intuitive for compensation benchmarking.
- Computing premium_24 and premium_25 separately and differencing them produces premium_expansion, revealing directional momentum without additional data joins.
- The function returns a dictionary rather than printing only, so results can be serialised directly to CSV in the output block.

Insight 3 — Stagnation Risk (insight_3_stagnation_risk)

Objective

Identify sectors where nominal wage growth from December 2024 to December 2025 falls below an assumed inflation rate of 3.5%, signalling that workers in those industries are losing real purchasing power.

Python Code

```
def insight_3_stagnation_risk():
    growth_data = []
    for idx, row in weekly_sa.iterrows():
        dec24 = row['Dec-24']
        dec25 = row['Dec-25']
        growth = ((dec25 - dec24) / dec24) * 100
        growth_data.append({'industry': row['Industry'],
                           'Growth %': growth, 'Dec-25 Weekly': dec25})
    df_growth = pd.DataFrame(growth_data).sort_values('Growth %')
    inflation_rate = 3.5
    underperformers = df_growth[df_growth['Growth %'] < inflation_rate]
    return df_growth
```

Key Design Choices

- Growth is multiplied by 100 inside the loop here (unlike Insight 1) because the inflation threshold comparison (< 3.5) requires percentage-scaled values, making the code self-consistent.
- Sorting ascending=True surfaces the weakest performers at the top of the DataFrame, matching the analytical intent.
- The inflation_rate constant (3.5%) is defined as a named variable rather than a magic number, making it easy to update when official CPI figures are finalised.
- Boolean mask df_growth[df_growth['Growth %'] < inflation_rate] counts underperformers without mutating the main DataFrame, preserving the full result for export.

Insight 4 — Skills Premium Paradox (insight_4_skills_premium)

Objective

Compare weekly earnings for all employees versus production workers in semiconductor and electronic components (NAICS 3344) to determine whether the white-collar / blue-collar earnings gap widened or narrowed between December 2024 and December 2025.

Python Code

```
def insight_4_skills_premium():
    naics = 3344
    all_emp = weekly_sa[weekly_sa['NAICS Code'] == naics].iloc[0]
    prod = weekly_prod_sa[weekly_prod_sa['NAICS Code'] == naics].iloc[0]
    premium_24 = ((all_emp['Dec-24'] - prod['Dec-24']) / prod['Dec-24']) * 100
    premium_25 = ((all_emp['Dec-25'] - prod['Dec-25']) / prod['Dec-25']) * 100
    gap_24 = all_emp['Dec-24'] - prod['Dec-24']
    gap_25 = all_emp['Dec-25'] - prod['Dec-25']
    premium_change = premium_25 - premium_24
```

```

    all_emp_growth = ((all_emp['Dec-25'] - all_emp['Dec-24']) / all_emp['Dec-24'])
    * 100
    prod_growth = ((prod['Dec-25'] - prod['Dec-24']) / prod['Dec-24']) * 100
    return {'premium_24': premium_24, 'premium_25': premium_25,
            'premium_change': premium_change}

```

Key Design Choices

- NAICS 3344 is used rather than 334413 because weekly_prod_sa contains production data at the broader subsector level; using a finer code would return no match.
- Computing both the absolute dollar gap (gap_24, gap_25) and the percentage premium provides complementary views: the dollar gap shows practical compensation distance; the percentage premium enables comparison across time.
- Separate growth rates for all_emp and production workers are calculated to diagnose whether convergence or divergence is driven by one group outpacing the other.
- The conditional print block ('Production workers are catching up!') is triggered only when prod_growth > all_emp_growth, providing interpretive context automatically without manual inspection.

The Paradox

If production worker wages grew faster than the all-employee average, the percentage premium narrows even while absolute dollar wages increase for both groups. This counter-intuitive result — higher wages AND a smaller gap — is the 'paradox' flagged in the function output.

Insight 5 — Recent Acceleration (insight_5_recent_acceleration)

Objective

Decompose the full-year (Dec-24 to Dec-25) wage growth of semiconductor device workers (NAICS 334413) into two sub-periods — the first ten months (Dec-24 to Oct-25) and the final two months (Oct-25 to Dec-25) — to detect whether wage momentum accelerated late in the year.

Python Code

```

def insight_5_recent_acceleration():
    naics = 334413
    semi = weekly_sa[weekly_sa['NAICS Code'] == naics].iloc[0]
    dec24_to_oct25 = ((semi['Oct-25'] - semi['Dec-24']) / semi['Dec-24']) * 100
    oct25_to_dec25 = ((semi['Dec-25'] - semi['Oct-25']) / semi['Oct-25']) * 100
    full_year = ((semi['Dec-25'] - semi['Dec-24']) / semi['Dec-24']) * 100
    oct_dec_ann = (oct25_to_dec25 / 2) * 12
    return {'q4_growth': oct25_to_dec25, 'full_year': full_year}

```

Key Design Choices

- October 2025 serves as the split point because it is the last data point before the terminal month, cleanly dividing the year into a 10-month baseline and a 2-month tail.

- Annualised rates are computed by dividing each sub-period growth by the number of months in that period then multiplying by 12, making the two windows directly comparable on an equal time basis.
- The conditional insight print ('Most growth happened early in year' vs 'Recent growth happened in Q4 2025') auto-classifies the pattern, flagging acceleration or front-loading without manual comparison.
- Returning the dictionary allows the acceleration flag to feed downstream dashboards or alerting logic.

Insight 6 — Real vs Nominal Wage Growth (insight_6_real_vs_nominal)

Objective

Adjust each sector's nominal Dec-24 to Dec-25 wage growth by a 3.5% inflation assumption to determine whether workers achieved genuine real purchasing-power gains. A boolean flag marks each sector as gaining (True) or losing (False) in real terms.

Python Code

```
def insight_6_real_vs_nominal():
    inflation_rate = 3.5
    for idx, row in weekly_sa.iterrows():
        industry      = row['Industry']
        nominal_growth = ((row['Dec-25'] - row['Dec-24']) / row['Dec-24']) * 100
        real_growth    = nominal_growth - inflation_rate
        real_indicator = 'True' if real_growth > 0 else 'False'
        print(f'{real_indicator} {industry[:60]}')
        print(f'Nominal: {nominal_growth:+.1f}% | Real: {real_growth:+.1f}%')
```

Note: The function prints only. The output block re-runs the same logic to build a serialisable DataFrame:

```
real_vs_nominal_rows = []
for idx, row in weekly_sa.iterrows():
    nominal = ((row['Dec-25'] - row['Dec-24']) / row['Dec-24']) * 100
    real     = nominal - inflation_rate
    real_vs_nominal_rows.append({
        'Industry': row['Industry'], 'NAICS Code': row['NAICS Code'],
        'Dec-24': row['Dec-24'], 'Dec-25': row['Dec-25'],
        'Nominal Growth %': round(nominal, 2),
        'Real Growth %': round(real, 2),
        'Real Wage Gain': real > 0
    })
pd.DataFrame(real_vs_nominal_rows).to_csv(output_dir +
'insight_6_real_vs_nominal.csv')
```

Key Design Choices

- Subtracting a single scalar `inflation_rate` from each sector's nominal growth rate applies Fisher approximation, which is appropriate here since differences between nominal and real rates are small.

- The boolean Real Wage Gain column (True/False) is stored natively as a Python bool, enabling direct groupby counting or filtering in any downstream analysis.
- Truncating the industry name to 60 characters (industry[:60]) in the print block prevents long NAICS descriptions from breaking the console layout.
- Duplicating the loop in the output block rather than refactoring insight_6 to return a DataFrame is a known technical debt; a future refactor should have the function return a DataFrame directly, eliminating the redundant pass.

Output Files

All results are written to a dedicated Findings subfolder on completion:

File	Source	Format
insight_1_wage_growth_rankings.csv	insight_1_communications_surge() return value	DataFrame (direct)
insight_2_semiconductor_premium.csv	insight_2_premium_gap() dict wrapped in DataFrame	Single-row DataFrame
insight_3_stagnation_risk.csv	insight_3_stagnation_risk() return value	DataFrame (direct)
insight_4_skills_premium.csv	insight_4_skills_premium() dict wrapped in DataFrame	Single-row DataFrame
insight_5_recent_acceleration.csv	insight_5_recent_acceleration() dict wrapped in DataFrame	Single-row DataFrame
insight_6_real_vs_nominal.csv	Rebuilt loop in output block	Multi-row DataFrame

Note: insight_6 is the only insight whose output is not derived directly from the function's return value; the loop is duplicated in the output block because insight_6_real_vs_nominal() only prints. This is flagged as a refactoring candidate.

Summary of Findings

The six insight functions collectively profile U.S. semiconductor-sector earnings dynamics across the Dec-24 to Dec-25 period:

- Wage growth rankings (Insight 1) identify which subsector led compensation growth, providing a competitive benchmark for talent strategy.
- The semiconductor device premium (Insight 2) quantifies both the absolute and relative earnings advantage of NAICS 334413 workers over the broader sector, and tracks whether that advantage is widening.
- Stagnation risk (Insight 3) flags sectors where nominal gains failed to keep pace with inflation, highlighting groups facing real wage erosion.

- The skills premium paradox (Insight 4) reveals whether production worker wages are converging toward white-collar earnings, and proposes three structural explanations (labour shortages, union negotiations, retention bonuses) when convergence is detected.
- Recent acceleration (Insight 5) decomposes full-year growth into early and late sub-periods, detecting whether momentum is building or fading into year-end.
- Real vs nominal analysis (Insight 6) applies a 3.5% inflation deflator to convert headline wage growth into purchasing-power-adjusted figures, identifying which sectors delivered genuine income gains.